



Padrão de Projeto MVC

Model · View · Controller

O que são Padrões de Projeto?

Soluções testadas e documentadas para problemas recorrentes no desenvolvimento de software

- Criados a partir do trabalho do Engenheiro Civil **Christopher Alexander** (década de 70) e adaptados para software.
- Um padrão é definido por quatro elementos essenciais: **Nome**, **Problema**, **Solução** e **Consequências**.
- **Benefícios gerais:** aumento de produtividade, uniformidade na estrutura, redução de complexidade e facilidade de manutenção.
- Usar padrões é considerado uma **boa prática** que ajuda a construir softwares confiáveis com arquiteturas testadas.

"Padrões de projeto descrevem soluções para problemas recorrentes no desenvolvimento de sistemas de software orientados a objetos."

Exemplos Conhecidos:

-  **Factory** (Criação de objetos)
-  **DAO** (Data Access Object)
-  **MVC** (Model-View-Controller)

Origem e Conceito do MVC

O MVC nasceu em 1979 e revolucionou a forma de organizar aplicações

Descrito originalmente no artigo *"Applications Programming in Smalltalk-80: How to use Model-View-Controller"*.

Objetivo principal: separar o projeto em três camadas independentes — Modelo, Visão e Controlador.

Pode ser aplicado em projetos **desktop, web e mobile**.

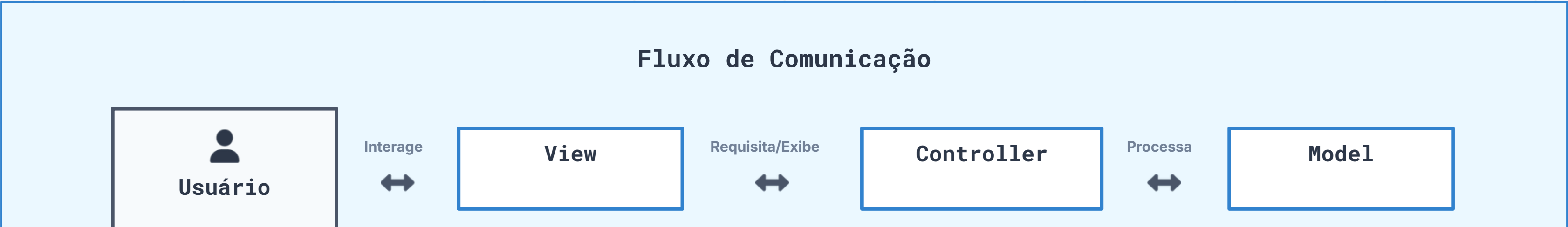
Essa separação atua diretamente em duas métricas fundamentais da engenharia de software: **Acoplamento** e **Coesão**.

Conceito	Definição e Impacto
Acoplamento	Grau em que uma classe conhece e depende de outra. ↓Baixo acoplamento é desejável
Coesão	Grau em que uma classe tem um único propósito bem definido. ↑Alta coesão é desejável

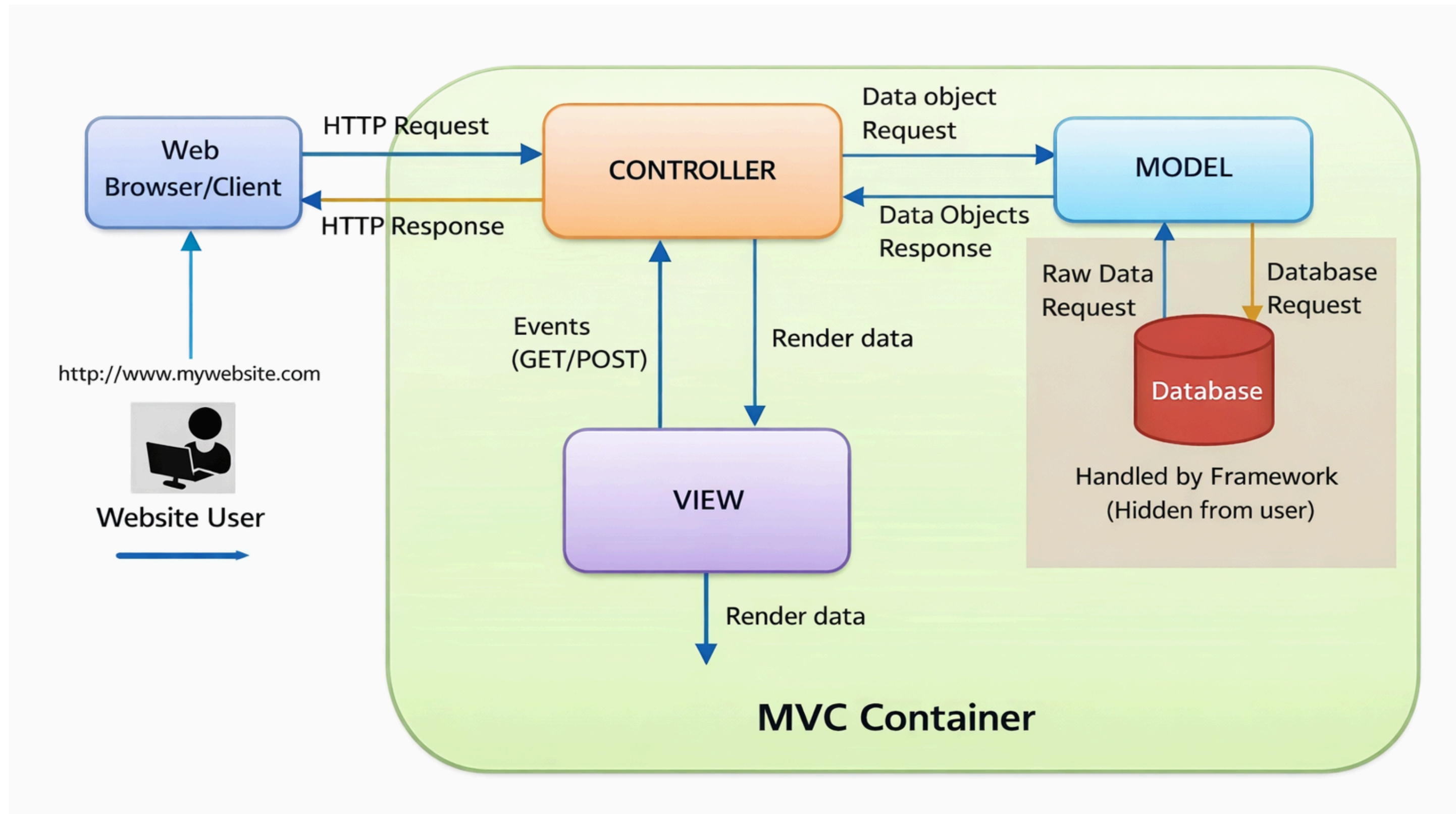
As Três Camadas do MVC

O MVC divide a aplicação em três camadas com responsabilidades bem definidas e independentes

Camada	Responsabilidade Principal
 Model	Lógica de negócio, regras, persistência de dados e entidades.
 View	Interface com o usuário — entrada e exibição de dados.
 Controller	Intermediário que conecta View e Model, gerenciando o fluxo de requisições e respostas.



As Três Camadas do MVC



Camada Model (Modelo)

O núcleo da aplicação: contém toda a lógica de negócio e o acesso aos dados

Recebe requisições vindas do **Controller** e retorna respostas processadas.

É responsável pelas **regras de negócio**, **persistência** (banco de dados, XML, TXT) e **classes de entidade**.

As operações de **CRUD** (Create, Retrieve, Update, Delete) são realizadas exclusivamente nesta camada.

Pode ser subdividido em pacotes especializados para manter a organização e a alta coesão.

Subdivisão de Pacotes

entity

Classes que representam tabelas do banco de dados (JavaBeans/POJOs).

dao

Classes de acesso a dados seguindo o padrão Data Access Object.

service

Classes que contêm as regras de negócio da aplicação.

"O Model é o responsável pelo motivo que a aplicação foi construída — ele é o núcleo da aplicação."

Camada View (Visão)

A View é a interface do usuário: exibe dados e captura entradas, sem conter lógica de negócio

Representa a parte do sistema com a qual o **usuário** interage diretamente.

Entrada de dados: formulários, campos de texto, seletores.

Saída de dados: tabelas, grids, resultados formatados.

Não contém lógica de negócio — todo processamento é feito pelo Model.

Pode assumir diferentes formas dependendo da plataforma.

Plataforma	Tecnologias de View
 Desktop	Swing, AWT, SWT (Java)
 Web	JSP, JSF, HTML/CSS, Angular, React
 Mobile	Activities (Android), SwiftUI (iOS), Flutter

★ A Grande Vantagem

É possível criar **múltiplas interfaces** (Desktop, Web, Mobile) para a mesma lógica de negócio (Model).

Camada Controller (Controlador)

O intermediário inteligente que organiza o fluxo entre **View** e **Model**

Recebe todos os **eventos gerados pelo usuário** na View (cliques, submissões de formulário).

Prepara e encaminha as requisições para o **Model** adequado.

Recebe as respostas do Model e as repassa para a **View** correspondente.

É o único componente que **conhece tanto a View quanto o Model**.

Torna possível o **reaproveitamento do Model** em outros sistemas, pois elimina a dependência direta entre View e Model.

Por que não comunicar View e Model diretamente?

A ligação direta geraria duas responsabilidades no código da interface: gerenciar a UI e lidar com a lógica de negócio — o que viola o princípio de **alta coesão**.

Vantagens e Desvantagens do MVC

Vantagens

Separação clara entre visualização e regras de negócio.

Manutenção e atualização do sistema mais fáceis.

Reaproveitamento de código — especialmente a camada Model em outros projetos.

Alterações na View não afetam as regras de negócio já implementadas.

Desenvolvimento, testes e manutenção de forma **isolada por camada**.

Desvantagens

Em sistemas de **baixa complexidade**, pode criar complexidade desnecessária.

Exige **disciplina** dos desenvolvedores na separação das camadas.

Requer um **tempo maior** para modelar o sistema inicialmente.

MVC na Prática: Exemplo Desktop e Web

O mesmo Model pode ser reutilizado em diferentes plataformas, demonstrando o poder do MVC

Camada	 Desktop (Swing)	 Web (Servlet/JSP)
View	<code>CalculoForm</code> (JFrame/Swing)	<code>calculoForm.jsp</code> + <code>resultadoForm.jsp</code>
Controller	<code>CalculoController</code> (método <code>executa()</code>)	<code>CalculoController</code> (método <code>service()</code>)
Model	 Reutilizado sem alterações (<code>CalculoService</code> + <code>CalculoDao</code> + <code>Calculo</code>)	

Exemplo: Sistema Financeiro de Cálculo de Juros

- A camada Model foi empacotada como biblioteca (`lib-model.jar`) e **reutilizada integralmente** no projeto web.
- Apenas View e Controller precisaram ser reescritos para a nova plataforma.
- Isso demonstra na prática o **ganho de produtividade** proporcionado pelo MVC.

Frameworks MVC e Conclusão

Frameworks MVC abstraem a complexidade e aceleram o desenvolvimento de aplicações web

Principais Frameworks Java	Destaque
JavaServer Faces (JSF)	Baseado em componentes, facilita o desenvolvimento da interface gráfica.
Struts 2	Suporte a anotações, Ajax e plugins.
VRaptor 3	Convenção sobre configuração, documentação em português.
Spring MVC	Configuração via anotações, parte do ecossistema Spring.

Conclusão

O padrão MVC é uma das práticas mais consolidadas no desenvolvimento de software moderno.

Ao separar claramente as responsabilidades de **dados, apresentação** e **controle de fluxo**, ele proporciona projetos mais organizados, fáceis de manter e com maior potencial de reuso — tornando-se uma competência essencial para qualquer desenvolvedor.